

Register the workflow. Let a controller keep it converged.

Green, Red and Blue reconcile a custom resource by running the same native workflow a launcher runs.

`redis-operator` supplies the controller; `doks` supplies its cluster.
Both packages ship in all three colours, verified live on 16
September 2026.

[Download the PDF](#) · [getcolors.ai](#)

A launcher run ends. A controller keeps observing.

One create

Reads `colors.yml`, converges, runs the checks, and exits.

Nobody observes the infrastructure after the exit code.

One controller

Reads a custom resource every poll and observes the infrastructure every `reconcileInterval`.

Converges only when the observation does not match the desired state.

The controller is an optional library in each SDK. It calls registered package functions and loads no code from a resource.

Three native runtimes, one controller contract

Colour	Runtime	Kubernetes SDK
Green	Clojure / Babashka	<code>green.kubernetes</code>
Red	TypeScript / Bun	<code>red/kubernetes</code>
Blue	Python / uv	<code>blue.kubernetes</code>

`./green`, `./red` or `./blue` selects the **CLI**. The `image` digest selects the **controller**. All three CLIs can operate the verified Red and Blue images.

Both packages share desired state and ownership keys, with byte-identical rendered artifacts. Run one lifecycle operation per profile at a time.

The controller in one slide

- The loop polls the custom resources with kubectl and coalesces queued work per resource.
- One process-wide lock per infrastructure state identity serializes whole workflows, across types and controller instances.
- The controller adds the finalizer `colors.getcolors.ai/infrastructure` before any effect.
- Every status write carries the `resourceVersion` of the snapshot it came from.

`spec.state` running · `spec.suspend` · `spec.reconcileInterval` 60s · `spec.deletionPolicy` Retain or Destroy · the `colors.getcolors.ai/reconcile-request` annotation

A package registers five callbacks

Callback	Returns
<code>validate</code>	Errors for an invalid configuration
<code>identity</code>	A stable name for the backend and the profile
<code>observe</code>	Existence, configuration match and readiness from a read-only look
<code>converge</code>	The package's converge workflow outcome
<code>delete</code>	The package's delete workflow outcome, repeatable

`converge` and `delete` call the same native workflow graphs a launcher runs.

One controller process. It implements no distributed leases or fencing.

redis-operator provisions one Droplet per RedisDeployment

The adapter

Imports the pinned `redis` package workflow. Only a confirmed provider 404 counts as absence; an authentication, provider, or state-read error is retried, never recreated.

The guards

`compute-prevent-destroy` stays on while the controller converges. Only an explicit `deletionPolicy: Destroy` lifts it.

Verbs: `build`, `create`, `check`, `rehearse`, `drill`, `restart`, `delete`. Redis 7.2 runs on a Droplet outside the cluster; the controller Pod runs the workflow.

The controller image and its volume

On the volume

A failed converge or delete is kept at `/data/work/<profile>/failures/`, 20 newest, every `COLORS_PAR_*` value masked to `***`.

SSH keys and convergence records persist on the volume. Green also persists its dependency caches; Red and Blue dependencies are installed in their images.

The image

`scripts/image.sh` builds `linux/amd64` from the checked-out commit and pushes it to the deployment's registry.

The deployment pins the printed digest as `image` in `colors.yml`.

doks is the cluster underneath

One managed cluster

DigitalOcean DOKS or Vultr VKE, named after the profile, through colors-compute's `managed-kubernetes` kind.

One optional registry

Deployment-owned, named after the profile, integrated with the cluster so DOKS places the pull Secret in every namespace.

Verbs: `build`, `create`, `check`, `kubeconfig`, `registry`, `delete`. A consumer deployment reads `.colors/<profile>/kubeconfig` by path and never commits it.

Historical Green cycle · 16 September 2026

Step	Result
<code>doks</code> create	Cluster and registry in 6 min 16 s. A second create found nothing to do in 13 s.
First operator create	Ready at generation 1
Image update	The controller rolled and ran no redundant create
Drill	Deleted Droplet 600954837; replacement 600968621 healthy after 5 min 21 s, resource UID unchanged
Rehearsal	Suspended at generation 2, resumed at generation 3
Restart	Passed on the second attempt; the new pod was up in about 20 s from cached dependencies
Destroy delete	The finalizer destroyed the Droplet, then the verb removed the namespace and the CRD
Shutdown	Cluster and registry destroyed. Account empty.

Later Red and Blue verification · 16 September 2026

Check	Observed result
DOKS	Red created cluster and registry; Blue converged the same resources; all three CLIs passed checks
Native controllers	Blue created Redis; Red adopted the same Droplet; both passed health, backup restore and restart
CLI compatibility	Green, Red and Blue CLIs all probed the Red image
Red recovery drill	601089393 → 601094147; old host absent, UID and generation unchanged, fresh authenticated write passed
Final handoff snapshot	Ready at generation 5, Red controller; cluster, registry, volume and Redis remained running

Service recovery does not restore deleted-host data automatically. Backup restoration was verified separately. Live provider: DigitalOcean; Vultr: fixture parity.

Four fixes from the original Green run

- `check` read `phase=Reconciling reason=Reconciling` on a healthy resource. It now polls every 5 s for up to 180 s through the periodic pass.
- A failed first Ansible attempt left nothing readable. The controller now retains masked failure logs, and `check` prints `failures retained: N`.
- A probe exec hit `Unexpected end of ZLIB input stream` in a pod still unpacking its toolchain. Every exec now waits for that pod's own `RedisDeployment controller running` line, and Green's caches live on the volume.
- The delete patch failed with `the server rejected our request due to an error in our request` because status writes had moved the `resourceVersion`. Patches now retry when only status moved.

A later failure was captured

Earlier Green creates sometimes failed at `redis/ansible`, then passed on retry. Those attempts had no retained diagnostic; their cause remains unproven.

During the later Red recovery drill, the retained log identified **Ubuntu's dpkg lock** on the replacement Droplet. The controller retried and converged without manual intervention.

Read masked diagnostics at `/data/work/<profile>/failures/<file>`.

This explains the captured later failure, not every earlier Ansible exit. Create and recovery waits tolerate Failed while the controller retries.

Find them in the catalog

Package recipes now carry a `shape`, one of single-node, multi-node, kubernetes, operator, or local. It renders as a chip on the card and a filter on `/skills`. Source pages list `artifacts`: the CRD, the controller image, the rendered manifests. A Context Skill inherits the shape of its companion package.

```
colour=red # or green or blue
# Run each install in its own deployment directory.
npx skills add getcolors/doks --skill package-doks-$colour
npx skills add getcolors/redis-operator --skill package-redis-operator-$colour
npx skills use getcolors/skills --skill redis-operator-kubernetes
```

[getcolors.ai/getcolors/doks](#) · [getcolors.ai/getcolors/redis-operator](#) · [redis-operator-kubernetes](#)

Online · Free · One hour

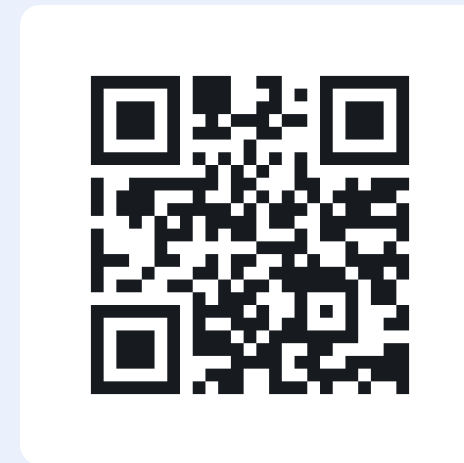
Join the Community Town Hall

Friday, 18 September 2026

17:00 CEST / 15:00 UTC

See a live provisioning demo, discuss the roadmap, and bring your infrastructure questions.

Register for the Town Hall



luma.com/ci9bek4c